

LAB EXPERIMENT – 2

Queries Using ANY, ALL, IN, EXISTS, NOT EXISTS, UNION, INTERSECT

Aim

To write and execute SQL queries using **subqueries** with the operators **ANY, ALL, IN, EXISTS, NOT EXISTS**, and set operations **UNION** and **INTERSECT**.

Software Required

- MySQL / Oracle / PostgreSQL
- Windows / Linux OS

Tables Used

STUDENT

```
CREATE TABLE STUDENT (  
    sid INT PRIMARY KEY,  
    sname VARCHAR(30),  
    dept VARCHAR(20),  
    marks INT  
);
```

COURSE

```
CREATE TABLE COURSE (  
    cid INT PRIMARY KEY,  
    cname VARCHAR(30),  
    sid INT,  
    FOREIGN KEY (sid) REFERENCES STUDENT(sid)  
);
```

INSERT STUDENT DATA:

STUDENT TABLE

```
INSERT INTO STUDENT VALUES (1,'Ravi','CSE',85);  
INSERT INTO STUDENT VALUES (2,'Anu','ECE',78);  
INSERT INTO STUDENT VALUES (3,'Kiran','CSE',92);  
INSERT INTO STUDENT VALUES (4,'Meena','IT',65);  
INSERT INTO STUDENT VALUES (5,'Suresh','ECE',88);
```

OUTPUT: STUDENT TABLE

sid sname dept marks

1	Ravi	CSE	85
2	Anu	ECE	78
3	Kiran	CSE	92
4	Meena	IT	65
5	Suresh	ECE	88

COURSE TABLE

```
INSERT INTO COURSE VALUES (101,'DBMS',1);  
INSERT INTO COURSE VALUES (102,'OS',3),  
INSERT INTO COURSE VALUES (103,'CN',5);
```

OUTPUT: COURSE TABLE**cid cname sid**

101	DBMS	1
102	OS	3
103	CN	5

Queries**1. IN Operator with Subquery**

Find students who are enrolled in any course.

```
SELECT sname  
FROM STUDENT  
WHERE sid IN (SELECT sid FROM COURSE);
```

Output:**sname**

Ravi
Kiran
Suresh

2. NOT IN Operator

Find students who are NOT enrolled in any course.

```
SELECT sname
FROM STUDENT
WHERE sid NOT IN (SELECT sid FROM COURSE);
```

Output:

sname

Anu

Meena

3. ANY Operator

Find students whose marks are greater than ANY student in ECE department.

```
SELECT sname
FROM STUDENT
WHERE marks > ANY (SELECT marks FROM STUDENT WHERE dept='ECE');
```

Output

sname

Ravi

Kiran

Suresh

4. ALL Operator

Find students whose marks are greater than ALL students in IT department.

```
SELECT sname
FROM STUDENT
WHERE marks > ALL (SELECT marks FROM STUDENT WHERE dept='IT');
```

Output

sname

Ravi

Anu

sname

Kiran

Suresh

5. EXISTS Operator

Find students who have enrolled in at least one course.

```
SELECT sname
FROM STUDENT S
WHERE EXISTS (
    SELECT *
    FROM COURSE C
    WHERE S.sid = C.sid
);
```

Output

sname

Ravi

Kiran

Suresh

6. NOT EXISTS Operator

Find students who have not enrolled in any course.

```
SELECT sname
FROM STUDENT S
WHERE NOT EXISTS (
    SELECT *
    FROM COURSE C
    WHERE S.sid = C.sid
);
```

Output

sname

sname

Anu

Meena

7. UNION Operator

Display students from CSE and ECE departments (without duplicates).

```
SELECT sname FROM STUDENT WHERE dept='CSE'  
UNION  
SELECT sname FROM STUDENT WHERE dept='ECE';
```

Output

sname

Ravi

Kiran

Anu

Suresh

8. INTERSECT Operator

Display students who scored above 80 AND are from CSE department.

```
SELECT sname FROM STUDENT WHERE marks > 80  
INTERSECT  
SELECT sname FROM STUDENT WHERE dept='CSE';
```

*(Note: INTERSECT is supported in Oracle/PostgreSQL.
For MySQL, use INNER JOIN.)*

Output

sname

Ravi

Kiran

Output

sname

Ravi

Kiran

Result

All queries using **ANY, ALL, IN, EXISTS, NOT EXISTS, UNION, INTERSECT** were executed successfully and the expected results were obtained.